CHEAT SHEET · GROUPE 1

Concevoir une API REST

Ressources · URIs · méthodes · pagination · versioning · erreurs | Sources : Fielding 2000, RFC 9110, OpenAPI 3.1, OWASP

LES 6 CONTRAINTES REST

Client-serveur	Séparer présentation et traitement.
Sans état	Chaque requête contient son contexte.
Cacheable	ETag, Cache-Control, Last-Modified.
Interface uniforme	Ressources, représentations, messages auto-descriptifs, HATEOAS.
Système en couches	Proxy, gateway, CDN, load balancer.
Code à la demande	Optionnel (JS côté client).

NOMMER LES RESSOURCES

- Noms au **pluriel** : `/tasks`, `/users`.
- **Minuscules**, tirets : `/task-lists`.
- **Pas de verbes** : `/tasks` et non `/getTasks`.
- Hiérarchie courte (2 niveaux max) : `/users/42/tasks`.
- Identifiants stables : `/tasks/42`.
- Actions spéciales : sous-ressource (`/tasks/42/comments`) ou `POST /tasks/42/complete` si vraiment nécessaire.

MAPPING CRUD

Action	HTTP + URI	Succès
Lister	`GET /tasks`	200
Lire	`GET /tasks/42`	200
Créer	`POST /tasks`	201
Remplacer	`PUT /tasks/42`	200/204
Modifier	`PATCH /tasks/42`	200/204
Supprimer	`DELETE /tasks/42`	204

PAGINATION / FILTRES / TRI

```
GET /api/v1/tasks
  ?done=false
  &priority=high
  &sort=-created_at
  &q=urgent
  &limit=20
  &offset=40
```

```
{
  "items": [ ... ],
  "total": 137,
  "limit": 20,
  "offset": 40,
  "next": "/api/v1/tasks?limit=20&offset=60",
  "prev": "/api/v1/tasks?limit=20&offset=20"
}
```

Curseur pour les gros volumes : `?cursor=eyJpZCI6NDJ9`.

VERSIONING

- **URI :** `/api/v1/tasks` — simple, visible.
- **En-tête :** `Accept: application/vnd.api.v1+json`.
- **Query :** `?version=1` — déconseillé.
- Ajouter un champ = non cassant ; supprimer/renommer = cassant.
- Annoncer la dépréciation : `Deprecation`, `Sunset`, changelog.

CODES DE STATUT À UTILISER

200	Lecture / mise à jour avec corps
201	Création + `Location`
204	Suppression / MAJ sans corps
304	Cache valide (conditionnel)
400	JSON mal formé / requête invalide
401	Token absent ou invalide
403	Droits insuffisants
404	Ressource inexistante
409	Doublon / conflit de version
422	Validation métier échouée
429	Quota dépassé + `Retry-After`

ERREUR NORMALISÉE (RFC 9457)

```
HTTP/1.1 422 Unprocessable Content
Content-Type: application/problem+json

{
  "type": "https://api.exemple.com/errors/validation",
  "title": "Échec de validation",
  "status": 422,
  "detail": "Le champ 'due_date' doit être futur.",
  "instance": "/api/v1/tasks",
  "request_id": "9b2f7c1e",
  "errors": [
    {"field":"due_date","code":"date_in_future"}
  ]
}
```

Un seul format · messages utiles · pas de fuite · `request_id`.

EXEMPLE COMPLET D'API

```
GET    /api/v1/tasks
GET    /api/v1/tasks/42
POST   /api/v1/tasks
PUT    /api/v1/tasks/42
PATCH  /api/v1/tasks/42
DELETE /api/v1/tasks/42

GET    /api/v1/tasks/42/comments
POST   /api/v1/tasks/42/comments

GET    /api/v1/users/7/tasks
GET    /health
GET    /docs       (Swagger UI)
GET    /openapi.json
```

HATEOAS

```
{
  "id": 42,
  "title": "Réviser REST",
  "done": false,
  "_links": {
    "self":   {"href": "/api/v1/tasks/42"},
    "update": {"href": "/api/v1/tasks/42", "method": "PATCH"},
    "delete": {"href": "/api/v1/tasks/42", "method": "DELETE"},
    "owner":  {"href": "/api/v1/users/7"}
  }
}
```

Maturité : 0 RPC · 1 ressources · 2 verbes HTTP · 3 HATEOAS.

CHECKLIST DE CONCEPTION

- Ressources au pluriel, URIs prévisibles.
- Méthodes et codes de statut corrects.
- Pagination, filtres, tri, recherche.
- Versioning et politique de dépréciation.
- Validation des entrées + erreurs normalisées.
- Cache (ETag, Cache-Control) et compression.
- Authentification, autorisation, rate limiting.
- Documentation OpenAPI + exemples.
- Idempotence des écritures sensibles.
- Tests et monitoring.

OPENAPI 3.1 : EXTRAIT

```yaml
openapi: 3.1.0
info:
  title: API Tâches
  version: 1.0.0
servers:
  - url: https://api.exemple.com/api/v1
paths:
  /tasks:
    get:
      summary: Lister les tâches
      parameters:
        - {name: done, in: query, schema: {type: boolean}}
        - {name: limit, in: query, schema: {type: integer, minimum: 1, maximum: 100}}
      responses:
        "200":
          description: Liste paginée
          content:
            application/json:
              schema: {$ref: "#/components/schemas/TaskList"}
components:
  schemas:
    Task:
      type: object
      required: [id, title, done]
      properties:
        id: {type: integer, format: int64}
        title: {type: string, minLength: 1, maxLength: 200}
        done: {type: boolean, default: false}
```

ANTI-PATTERNS À ÉVITER

- `/getTasks`, `/createTask` : verbes dans l'URI.
- `POST /tasks/42/delete` au lieu de `DELETE`.
- Renvoyer `200` avec `{"error": ...}`.
- `GET` qui modifie des données.
- Pas de pagination sur les collections.
- Champs sensibles dans les réponses.
- Versioning absent et breaking changes silencieux.

SÉCURITÉ REST

- HTTPS + authentification sur chaque endpoint.
- Vérifier la propriété (BOLA) et les rôles.
- Valider les entrées, limiter la taille.
- Rate limiting et quotas par client.
- Ne pas exposer d'IDs prévisibles sans contrôle.
- Logs d'audit, `request_id`, alerting.